

BA05

**DESKRIPSI PERANCANGAN PERANGKAT LUNAK
KOSTOPIA**

*Disusun untuk memenuhi Tugas Besar
Mata Kuliah Implementasi dan Pengujian Perangkat Lunak (CAK3BAB3)*



Disusun oleh:

Aryo Dwipa Maulana	-	103012400081
Faizul 'Afiat	-	103012400356
Fikri Rahmatullah	-	103012400067
Ravananda Nadzhief Atthalah	-	103012400117
Raynar Khalid Mahardika Sumarno	-	103012430017

S1 INFORMATIKA UNIVERSITAS TELKOM BANDUNG	Nomor Dokumen		Nomor Halaman
	<i>BA05-DPPL-01</i>		<i>1/21</i>
	Revisi		30/09/2026

DAFTAR PERUBAHAN

Revisi	Deskripsi
A	
B	
C	
D	
E	

INDEX	-	A	B	C	D	E
TGL						
Ditulis oleh						
Diperiksa oleh						
Disetujui oleh						

DAFTAR ISI

BAB I PENDAHULUAN	5
1.1 Tujuan Penulisan Dokumen.....	5
1.2 Lingkup Proyek	5
1.3 Definisi, Akronim, dan Singkatan	5
BAB II PERANCANGAN ARSITEKTUR SISTEM	7
2.1 Arsitektur Perangkat Lunak	7
2.2 Topologi dan Alur Komunikasi Sistem.....	8
2.3 Infrastruktur Pendukung dan Lingkungan Pengembangan	9
BAB III PERANCANGAN DATA DAN PROSES	11
3.1 Diagram Hubungan Entitas (<i>Entity Relationship Diagram - ERD</i>)	11
3.2 Skema Relasi Tabel Basis Data (PostgreSQL)	11
3.3 Diagram Alir Data (<i>Data Flow Diagram - DFD</i>)	14
3.3.1 <i>DFD Level 0</i> (Diagram Konteks)	14
3.3.2 <i>DFD Level 1</i> (Model Proses Utama)	15
3.3.3 <i>DFD Level 2</i> (Penjabaran Rinci)	16
BAB IV PERANCANGAN RINCI.....	18
4.1 Perancangan Antarmuka Pemrograman Aplikasi (<i>RESTful API Endpoints</i>).....	18
4.2 Perancangan Struktur Data dan Skema Validasi (SQLAlchemy & Pydantic).....	18
4.3 Perancangan Interaksi Proses (<i>Sequence Diagram</i>)	19
BAB V KESIMPULAN	21

DAFTAR GAMBAR

Gambar 1.3.1 System Architecture Diagram.....	7
Gambar 3.1.1 Entity Relationship Diagram	11
Gambar 3.3.1 Data Flow Diagram Level 0	14
Gambar 3.3.2 Data Flow Diagram Level 1	15
Gambar 3.3.3 Data Flow Diagram Level 2	16
Gambar 3.3.4 Data Flow Diagram Level 2	17
Gambar 4.3.1 Sequence Diagram.....	19

BAB I

PENDAHULUAN

1.1 Tujuan Penulisan Dokumen

Dokumen perancangan perangkat lunak ini bertujuan untuk menyajikan cetak biru teknis (*technical blueprint*) dan arsitektur sistem secara komprehensif bagi para pengembang (*developers*) dan penguji (*QA*). Dokumen ini menerjemahkan spesifikasi kebutuhan fungsional dari dokumen SKPL ke dalam bentuk rancangan teknis implementasi sistem, termasuk struktur basis data relasional, perancangan antarmuka pemrograman aplikasi (*API endpoints*), serta mekanisme alur komunikasi data antar komponen perangkat lunak pada aplikasi Kostopia.

1.2 Lingkup Proyek

Aplikasi Kostopia dirancang sebagai sistem manajemen rumah kos berbasis web dinamis (*responsive web application*) yang mengintegrasikan dua peran pengguna (*multi-role*), yaitu **Pemilik Kos** dan **Penghuni Kos**. Lingkup teknis perancangan sistem ini mencakup:

1. **Modul Pengelolaan Keuangan:** Otomatisasi pembuatan tagihan, unggah bukti bayar, validasi transaksi, serta rekapitulasi laporan keuangan.
2. **Modul Akun & Parkir:** Manajemen profil identitas serta pencatatan nomor kendaraan penghuni untuk pengawasan area parkir.
3. **Modul Aspirasi & Perizinan:** Pengelolaan tiket keluhan fasilitas serta alur pengajuan dan persetujuan surat izin keluar masuk penghuni.
4. **Modul Informasi & Aturan Dinamis:** Penyiaran pengumuman (*broadcast*), pengelolaan tata tertib kos secara kustom, pencarian barang hilang (*Lost & Found*), serta modul evaluasi melalui ulasan dan penilaian (*rating & feedback*).

1.3 Definisi, Akronim, dan Singkatan

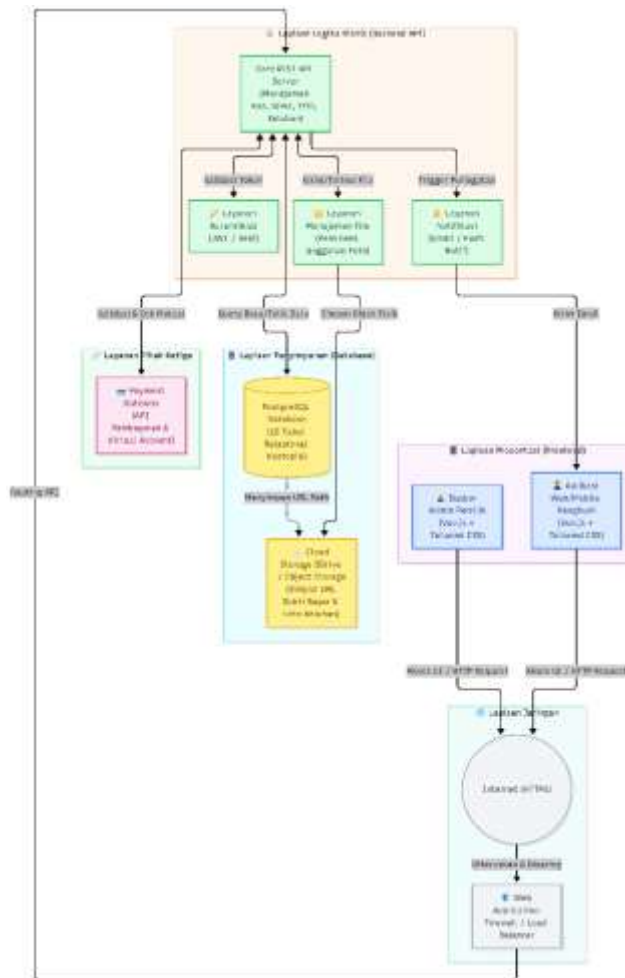
Untuk menghindari kesalahpahaman penafsiran teknis dalam dokumen ini, berikut adalah daftar istilah dan singkatan yang digunakan:

1. **API (*Application Programming Interface*):** Antarmuka yang menjembatani komunikasi antara sistem sisi depan (*frontend*) dan sisi belakang (*backend*).
2. **FastAPI:** Kerangka kerja (*framework*) berbasis Python yang digunakan untuk membangun API dengan kinerja tinggi secara asinkron.
3. **Vue.js:** Kerangka kerja JavaScript yang digunakan untuk membangun antarmuka pengguna pada sisi klien secara interaktif.

Jurusan Informatika Telkom	BA05-DPPL-01	Halaman 5 dari 21
Dokumen ini dan informasi yang dimilikinya adalah milik Jurusan Informatika-Telkom dan bersifat rahasia. Dilarang me-reproduksi dokumen ini tanpa diketahui oleh Jurusan Informatika-Telkom.		

4. **PostgreSQL:** Sistem manajemen basis data relasional (*RDBMS*) yang digunakan untuk menyimpan seluruh data terstruktur aplikasi.
5. **ORM (*Object-Relational Mapping*):** Teknik pemrograman untuk memetakan tabel basis data relasional ke dalam bentuk kelas objek di dalam bahasa Python menggunakan SQLAlchemy.
6. **JSON (*JavaScript Object Notation*):** Format standar berbasis teks yang digunakan untuk pertukaran data antara peladen dan klien.
7. **JWT (*JSON Web Token*):** Standar industri untuk mengamankan proses autentikasi sesi masuk pengguna ke dalam sistem.

BAB II PERANCANGAN ARSITEKTUR SISTEM



Gambar 1.3.1 System Architecture Diagram

2.1 Arsitektur Perangkat Lunak

Aplikasi Kostopia dirancang menggunakan pola arsitektur **Client-Server** berbasis layanan terpusat (*Centralized Service-Oriented Architecture*). Pola ini memisahkan secara tegas antara komponen antarmuka pengguna di sisi klien (*frontend*) dan komponen pengolahan data serta logika bisnis di sisi peladen (*backend*).

1. **Sisi Klien (*Frontend*):** Dibangun menggunakan kerangka kerja Vue.js, berjalan langsung pada peramban web atau perangkat seluler pengguna, serta bertanggung jawab untuk merender tampilan antarmuka secara dinamis bagi Pemilik Kos dan Penghuni Kos.
2. **Sisi Peladen (*Backend*):** Dibangun menggunakan kerangka kerja FastAPI berbasis bahasa Python, yang bertindak sebagai pusat pemrosesan logika aplikasi, manajemen autentikasi sesi, serta pengatur aturan bisnis.

3. **Penyimpanan Data (*Database*):** Menggunakan PostgreSQL sebagai sistem basis data relasional utama untuk menampung seluruh rekam jejak informasi secara aman dan terstruktur.

2.2 Topologi dan Alur Komunikasi Sistem

Komunikasi data antar komponen dalam ekosistem sistem Kostopia berjalan melalui mekanisme berikut:

1. **Permintaan Pengguna (*Client Request*):** Ketika pengguna (pemilik atau penghuni) melakukan suatu tindakan pada antarmuka Vue.js (seperti menekan tombol kirim pembayaran atau membuka menu izin), aplikasi klien akan mengirimkan permintaan HTTP/HTTPS (*RESTful API request*) ke peladen.
2. **Autentikasi dan Validasi Token:** Setiap permintaan data yang memerlukan hak akses khusus harus menyertakan token keamanan berbasis JWT (*JSON Web Token*) pada bagian header permintaan. FastAPI akan memvalidasi token tersebut sebelum memproses instruksi lebih lanjut.
3. **Eksekusi Logika Bisnis:** FastAPI menerima permintaan, menyaring dan memvalidasi struktur data masukan menggunakan modul skema Pydantic, lalu mengeksekusi fungsi logika bisnis yang sesuai.

4. **Interaksi Basis Data:** Melalui perantara ORM (SQLAlchemy), FastAPI melakukan operasi kueri (*query*) baca atau tulis ke dalam basis data PostgreSQL.
5. **Respon Sistem (Server Response):** Setelah data diproses, peladen mengembalikan hasil akhir ke aplikasi Vue.js dalam format data terstruktur JSON, yang kemudian langsung diterjemahkan dan ditampilkan kembali ke layar pengguna secara *real-time*.

2.3 Infrastruktur Pendukung dan Lingkungan Pengembangan

Untuk memastikan kelancaran proses pembangunan, pengelolaan kode, sumber, pengajian, serta operasional harian aplikasi Kostopia, sistem didukung oleh infrastruktur perangkat lunak dan layanan eksternal dengan rincian sebagai berikut:

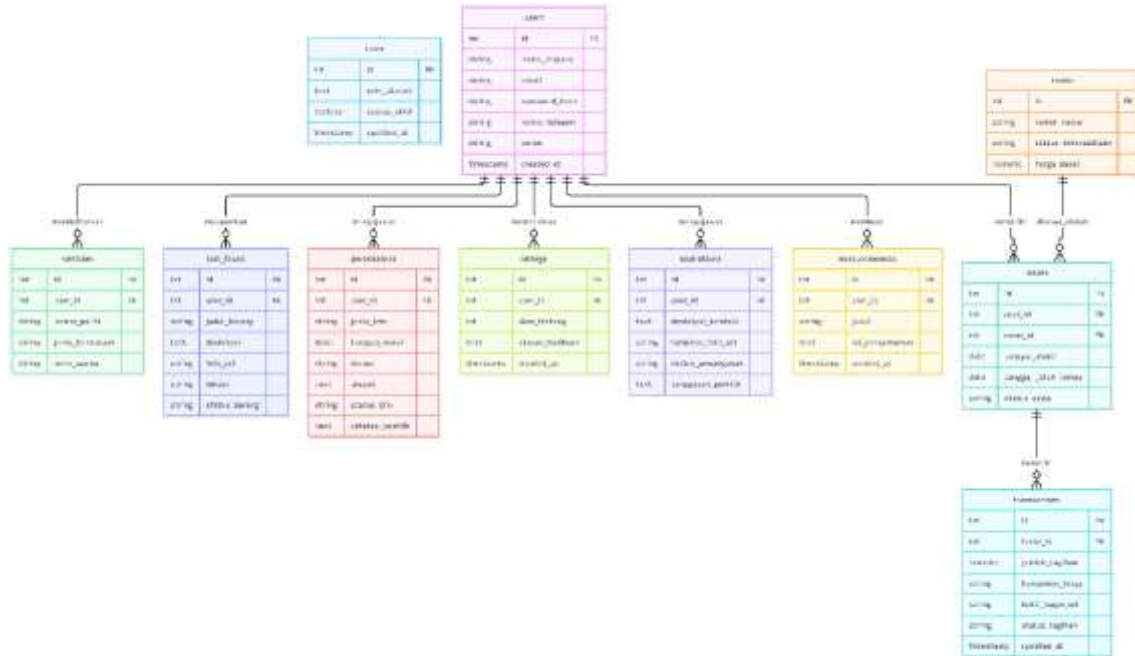
1. **Pengelolaan Kode Sumber dan Kolaborasi (Version Control System)**
 - **GitHub:** Digunakan sebagai usat repositori penyimpanan kode sumber utama (*source code repository*) untuk memisahkan modul *server* FastAPI (*backend*) dan antarmuka Vue.js (*frontend*).
 - **Strategi Cabang (Branching Strategy):** Penerapan alur kerja kolaborasi berbasis cabang fitur (*feature branching*) dan penarikan permintaan (*pull request*) guna menjaga integritas kode utama serta memudahkan pelacakan riwayat perubahan (*changelog*) antar pengembang.
2. **Lingkungan Bahasa dan Manajemen Pustaka (Runtime & Package Management)**
 - **Python (Versi 3.10+):** Sebagai bahasa pemrograman inti di sisi *server* yang mendukung eksekusi kerangka kerja FastAPI secara asinkron.
 - **PIP & Poetry:** Alat pengelola dependensi pustaka Python untuk memastikan konsistensi versi modul seperti *SQLAlchemy*, *Pydantic*, dan pustaka enkripsi token JWT.
 - **Node.js & Vite:** Lingkungan eksekusi JavaScript dan perangkat pembangun (*build tool*) berkinerja tinggi yang digunakan untuk mengompilasi dan menjalankan antar muka berbasis kerangka kerja Vue.js di sisi klien.
3. **Layanan Penyimpanan Berkas Eksternal (Cloud File Storage)**
 - **Google Drive API:** Diintegrasikan sebagai media penyimpanan awan (*cloud storage*) eksternal untuk menampung arsip berkas digital berukuran besar yang diunggah oleh pengguna.
 - Berkas-berkas yang dikelola mencakup lampiran bukti transfer pembayaran sewa, foto dokumen perizinan, serta foto laporan kerusakan fasilitas pada modul aspirasi (*Lost & Found* dan keluhan). Basis data PostgreSQL secara spesifik hanya akan menyimpan tautan URL (*file path/url*) menuju berkas tersebut untuk menjaga efisiensi kapasitas basis data.

4. Manajemen Konfigurasi dan Keamanan Sistem (*Environment & Security*)

- **Variabel Lingkungan (.env):** Berfungsi untuk memisahkan data konfigurasi sensitif dari kode sumber utama (seperti *Database URL*, *Secret Key* autentikasi JWT, dan token akses API eksternal) guna menjamin aspek keamanan sistem dan mencegah kebocoran kredensial (*security best practices*).
- **Instalasi Native / Lokal:** Lingkungan pengembangan dijalankan langsung pada sistem operasi pengembangan dengan menghubungkan *server* lokal FastAPI langsung ke instans basis data relasional PostgreSQL yang terpasang pada komputer pengembangan.

BAB III PERANCANGAN DATA DAN PROSES

3.1 Diagram Hubungan Entitas (*Entity Relationship Diagram - ERD*)



Gambar 3.1.1 Entity Relationship Diagram

Perancangan basis data aplikasi Kostopia menggunakan model relasional untuk memastikan integritas data, ketersediaan informasi, dan efisiensi penyimpanan terkelola dengan baik. Berdasarkan kebutuhan fungsional sistem yang melibatkan dua peran pengguna, basis data dirancang untuk menampung entitas utama yang saling berelasi, meliputi data pengguna, kamar, masa sewa, transaksi keuangan, kendaraan parkir, aspirasi, perizinan, pengumuman, aturan dinamis, barang hilang (*Lost & Found*), serta ulasan penilaian (*rating & feedback*).

3.2 Skema Relasi Tabel Basis Data (PostgreSQL)

Berikut adalah rincian struktur rancangan tabel di dalam basis data PostgreSQL:

1. Tabel Pengguna (*users*):

- Berfungsi untuk menyimpan informasi kredensial akun dan profil dasar pengguna.
- Atribut utama: `id` (Primary Key), `nama_lengkap`, `email`, `password_hash`, `nomor_telepon`, `peran` (Pemilik atau Penghuni), dan `created_at`.

2. Tabel Kamar & Sewa (rooms & leases):

- Berfungsi untuk mengelola ketersediaan kamar serta masa aktif sewa penghuni.
- Atribut utama tabel rooms: *id* (Primary Key), *nomor_kamar*, *status_ketersediaan*, dan *harga_dasar*.
- Atribut utama tabel leases: *id* (Primary Key), *user_id* (Foreign Key ke tabel *users*), *room_id* (Foreign Key ke tabel *rooms*), *tanggal_mulai*, *tanggal_jatuh_tempo*, dan *status_sewa*.

3. Tabel Keuangan / Transaksi (transactions):

- Berfungsi untuk mencatat rincian tagihan bulanan dan unggahan bukti bayar.
- Atribut utama: *id* (Primary Key), *lease_id* (Foreign Key), *jumlah_tagihan*, *komponen_biaya*, *bukti_bayar_url*, *status_tagihan* (Belum Bayar, Menunggu Verifikasi, Lunas), dan *updated_at*.

4. Tabel Kendaraan Parkir (vehicles):

- Berfungsi untuk mencatat identitas kendaraan penghuni yang terintegrasi dengan area parkir.
- Atribut utama: *id* (Primary Key), *user_id* (Foreign Key), *nomor_polisi*, *jenis_kendaraan*, serta *merk_warna*.

5. Tabel Aspirasi & Keluhan (aspirations):

- Berfungsi untuk menampung tiket laporan kerusakan fasilitas atau saran dari penghuni.
- Atribut utama: *id* (Primary Key), *user_id* (Foreign Key), *deskripsi_kendala*, *lampiran_foto_url*, *status_penanganan* (Menunggu, Sedang Diproses, Selesai), dan *tanggapan_pemilik*.

6. Tabel Perizinan (permissions):

- Berfungsi untuk mengelola alur pengajuan izin keluar masuk penghuni.
- Atribut utama: *id* (Primary Key), *user_id* (Foreign Key), *jenis_izin*, *tanggal_mulai*, *durasi*, *alasan*, *status_izin* (Pending, Disetujui, Ditolak), dan *catatan_pemilik*.

7. Tabel Pengumuman (announcements):

- Berfungsi untuk menyimpan riwayat siaran pesan penting dari pemilik kos.
- Atribut utama: `id` (Primary Key), `user_id` (Foreign Key sebagai pembuat), `judul`, `isi_pengumuman`, dan `created_at`.

8. Tabel Aturan Dinamis (rules):

- Berfungsi untuk menyimpan poin-poin peraturan tata tertib kos yang kustom.
- Atribut utama: `id` (Primary Key), `pribadi_poin_aturan`, `status_aktif`, dan `updated_at`.

9. Tabel Barang Hilang (lost_found):

- Berfungsi untuk memuat papan informasi barang hilang atau temuan di area kos.
- Atribut utama: `id` (Primary Key), `user_id` (Foreign Key pelapor), `judul_barang`, `deskripsi`, `foto_url`, `lokasi`, dan `status_barang` (Aktif/Belum Ditemukan, Selesai).

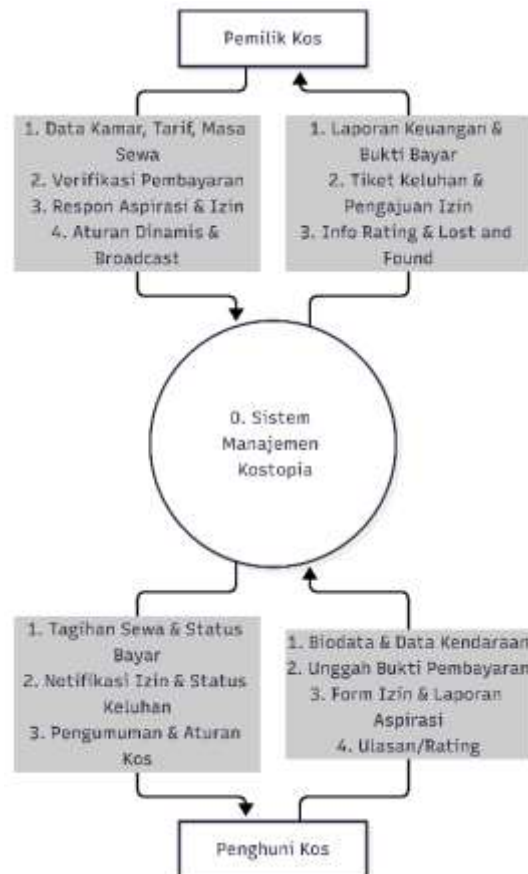
10. Tabel Penilaian & Ulasan (ratings):

- Berfungsi untuk menampung skor rating bintang dan ulasan fasilitas kos.
- Atribut utama: `id` (Primary Key), `user_id` (Foreign Key), `skor_bintang`, `ulasan_feedback`, dan `created_at`.

3.3 Diagram Alir Data (Data Flow Diagram - DFD)

Diagram Alir Data digunakan untuk memvisualisasikan bagaimana data mengalir melalui sistem aplikasi Kostopia dari entitas luar, melewati proses pengolahan di dalam sistem, hingga disimpan ke dalam basis data.

3.3.1 DFD Level 0 (Diagram Konteks)

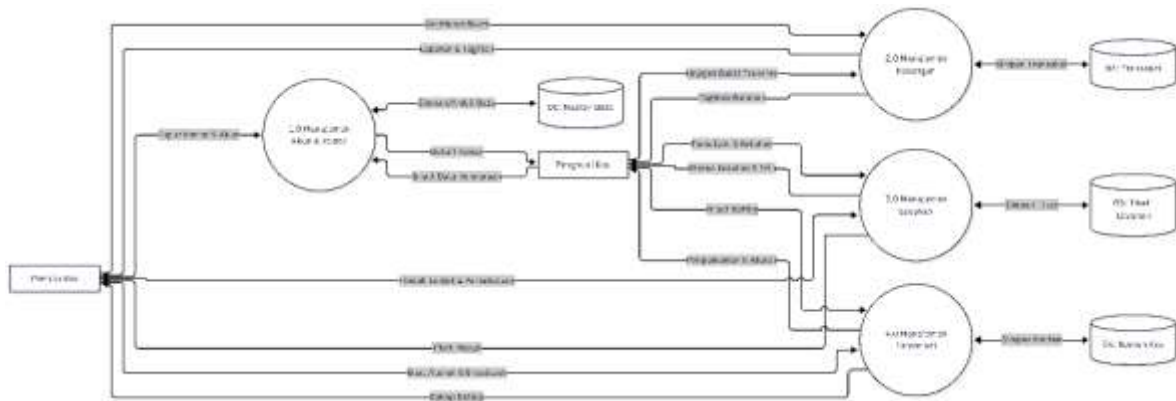


Gambar 3.3.1 Data Flow Diagram Level 0

Diagram Konteks memodelkan aplikasi Kostopia sebagai sistem tunggal (*macro*) yang berinteraksi langsung dengan dua entitas luar:

1. **Pemilik Kos:** Mengirimkan data tarif, validasi pembayaran, keputusan izin, siaran pengumuman, aturan dinamis, serta memantau rekapitulasi keuangan dan ulasan.
2. **Penghuni Kos:** Mengirimkan data profil, nomor kendaraan, unggahan bukti bayar, laporan aspirasi, pengajuan izin, laporan barang hilang, serta ulasan penilaian.
3. **Sistem Kostopia:** Menerima seluruh masukan tersebut, memproses logika bisnis, dan mengembalikan data keluaran berupa status tagihan, notifikasi izin, daftar pengumuman, serta rekapitulasi laporan ke masing-masing entitas.

3.3.2 DFD Level 1 (Model Proses Utama)



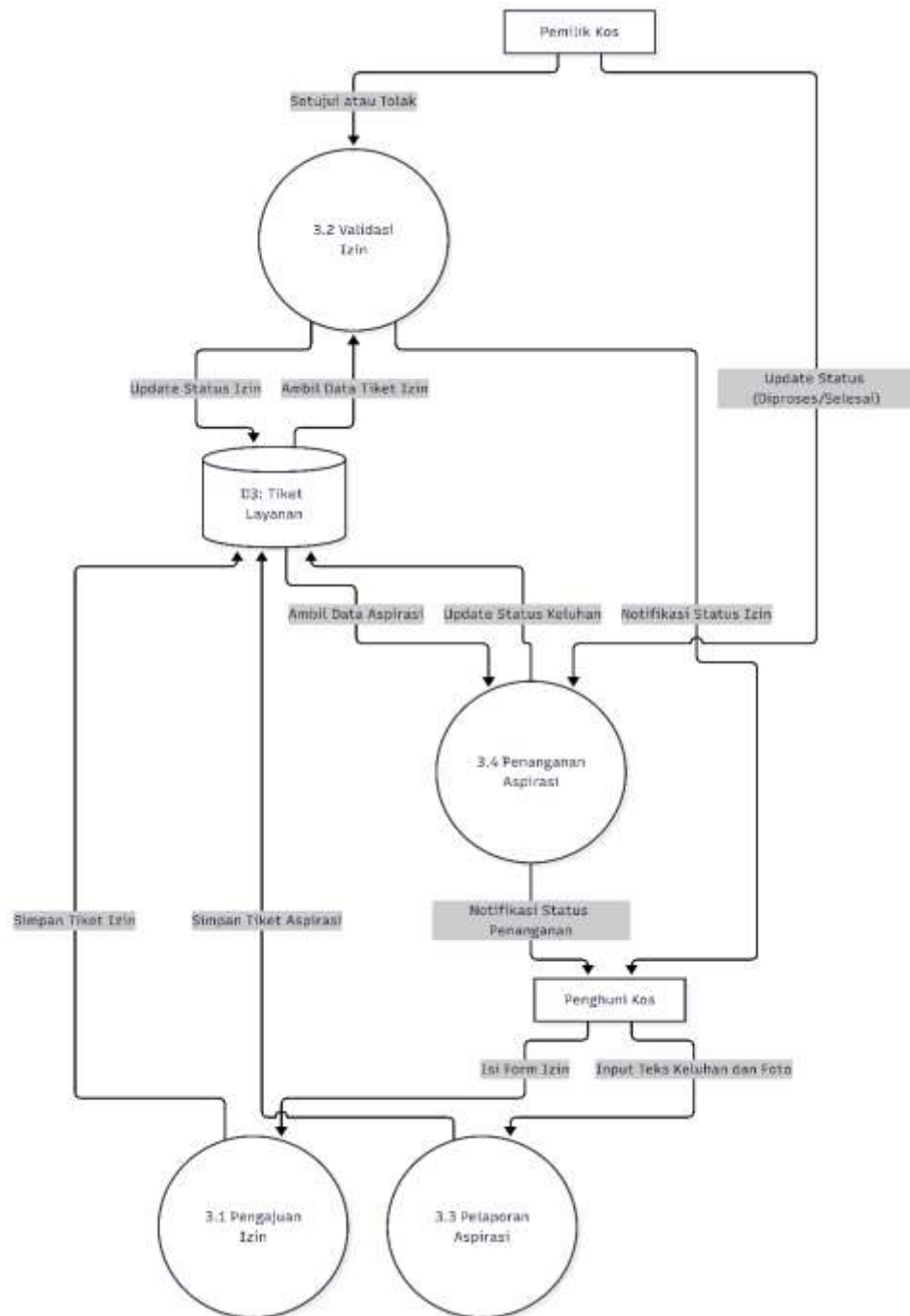
Gambar 3.3.2 Data Flow Diagram Level 1

Pada *DFD Level 1*, sistem aplikasi Kostopia dijabarkan ke dalam proses utama yang saling terhubung dengan penyimpanan data (*data store*):

1. **Proses 1.0 (Manajemen Akun & Autentikasi):** Mengelola data pendaftaran, masuk (*login*), pengelolaan profil, serta pembaruan status penghuni.
2. **Proses 2.0 (Pengelolaan Keuangan & Parkiran):** Mengatur penetapan tarif sewa, pembuatan tagihan otomatis, unggah bukti bayar, validasi transaksi, rekapitulasi, serta pendataan kendaraan parkir.
3. **Proses 3.0 (Pengelolaan Aspirasi & Perizinan):** Mengelola alur pengiriman tiket keluhan fasilitas oleh penghuni serta pemrosesan surat izin keluar masuk.
4. **Proyek 4.0 (Layanan Informasi & Evaluasi):** Mengatur siaran pengumuman, pengelolaan aturan dinamis kos, pengelolaan papan barang hilang (*Lost & Found*), serta penyimpanan ulasan penilaian layanan.

3.3.3 DFD Level 2 (Penjabaran Rinci)

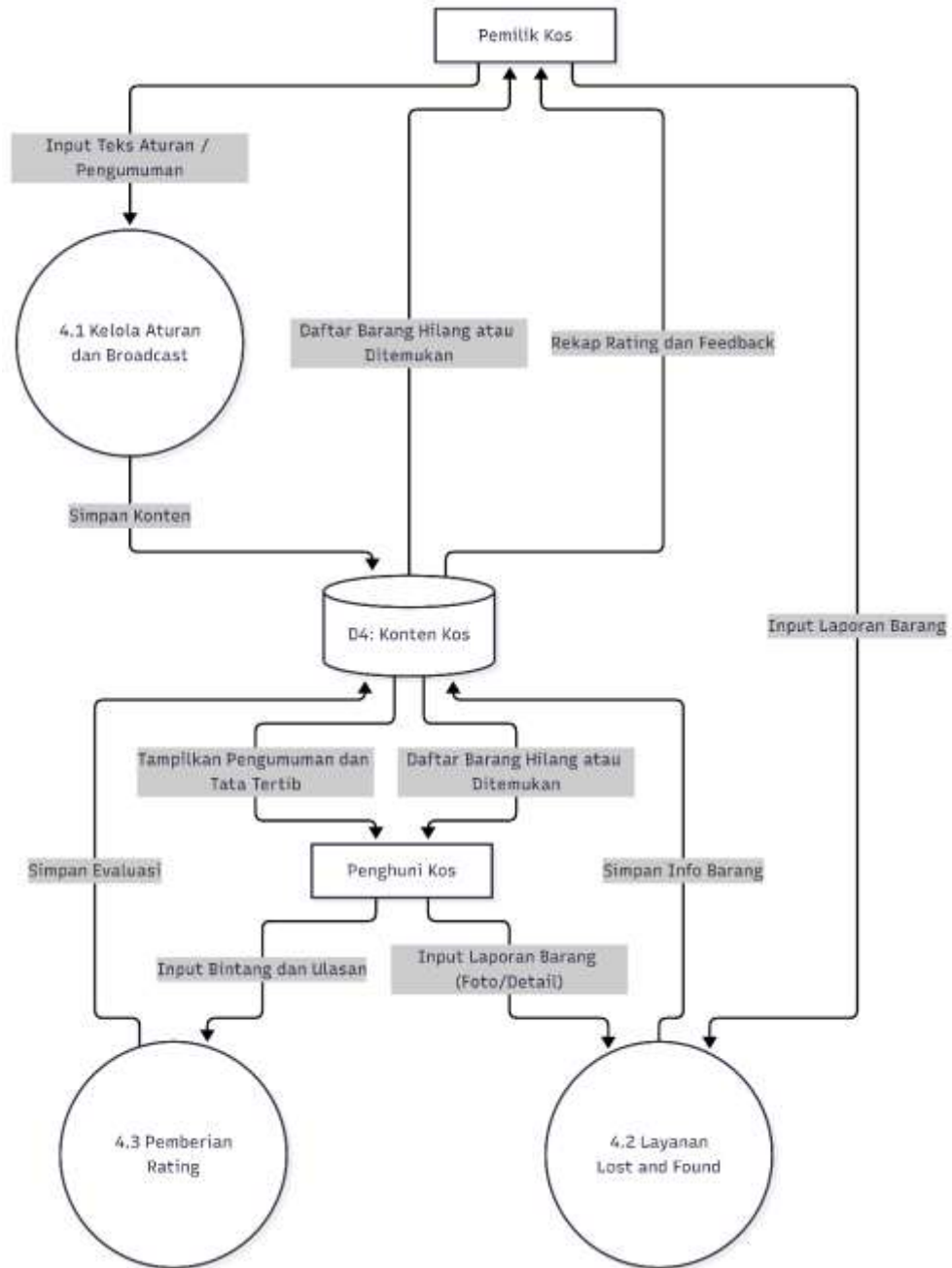
- **DFD Level 2 Proses 3.0 (Pengelolaan Aspirasi & Perizinan)**



Gambar 3.3.3 Data Flow Diagram Level 2

Menjabarkan lebih detail sub-proses pengisian formulir aspirasi, penyimpanan tiket ke basis data aspirasi, pengiriman notifikasi ke dasbor pemilik, hingga pembaruan status penanganan dan tanggapan. Serta merinci sub-proses pengajuan izin, validasi dokumen oleh pemilik, hingga pengiriman pemberitahuan status izin kepada penghuni.

- **DFD Level 2 Proses 4.0 (Layanan Informasi & Evaluasi)**



Gambar 3.3.4 Data Flow Diagram Level 2

Menjabarkan sub-proses penyiaran pengumuman *real-time*, pembaruan poin tata tertib kostum, pengelolaan status barang pada papan *Lost & Found*, hingga proses kalkulasi akumulasi ulasan bintang dan *feedback* dari penghuni.

BAB IV

PERANCANGAN RINCI

4.1 Perancangan Antarmuka Pemrograman Aplikasi (*RESTful API Endpoints*)

Bagian ini mendefinisikan rincian antarmuka pemrograman aplikasi yang dibangun menggunakan kerangka kerja FastAPI di sisi *server*. Komunikasi antara antarmuka Vue.js dan *server* FastAPI dilakukan menggunakan format data terstruktur. JSON melalui protokol HTTP dengan rincian modul rute utama sebagai berikut:

1. **Modul Autentikasi (/api/v1/auth):** Menyediakan titik akhir untuk proses masuk pengguna (*login*), pendaftaran akun baru, penyegaran sesi token, dan verifikasi token akses berbasis JWT (*JSON Web Token*).
2. **Modul Pengelolaan Keuangan (api/v1/transactions & /leases):** Menyediakan titik akhir untuk menghasilkan tagihan sewa bulanan secara otomatis, mengunggah berkas bukti transfer pembayaran, validasi status transaksi oleh pemilik kos, serta rekapitulasi laporan keuangan.
3. **Modul Akun dan Parkir (/api/v1/users & /vehicles):** Menyediakan titik akhir untuk memperbarui profil identitas pengguna serta mengelola pendaftaran nomor polisi kendaraan penghuni untuk pengawasan area parkir.
4. **Modul Aspirasi dan Perizinan (/api/v1/aspirations & /permissions):** Menyediakan titik akhir untuk mengirimkan tiket keluhan fasilitas, memperbarui status penanganan, serta mengajukan dan memvalidasi surat izin keluar masuk penghuni.
5. **Modul Layanan Informasi & Evaluasi (/api/v1/announcements, /rules, /lost-found, /ratings):** Menyediakan titik akhir untuk siaran pengumuman, pembaruan tata tertib dinamis, pengelolaan papan informasi barang hilang, serta pengiriman ulasan penilaian layanan kos.

4.2 Perancangan Struktur Data dan Skema Validasi (SQLAlchemy & Pydantic)

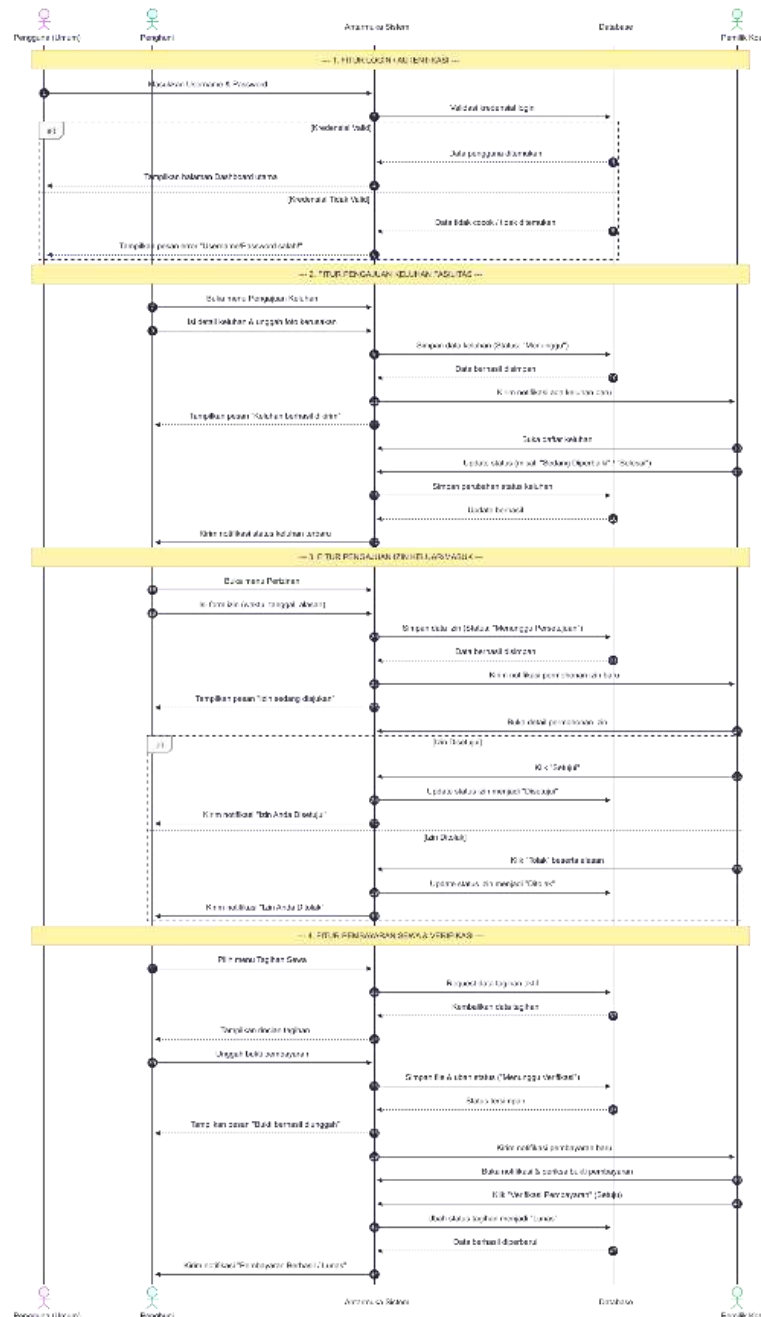
Perancangan rincian struktur data di sisi *server* berfokus pada pemetaan basis data relasional dan mekanisme validasi masukan fungsional:

1. **Pemetaan Relasi Objek (SQLAlchemy ORM Models):** Mendefinisikan struktur model tabel basis data PostgreSQL ke dalam entitas model Python (seperti model User, Room, Lease, Transaction, Vehicle, Aspiration, Permission, Announcement, Rule, LostFound, dan Rating) untuk menangani operasi kueri basis data secara terstruktur.

Jurusan Informatika Telkom	BA05-DPPL-01	Halaman 18 dari 21
Dokumen ini dan informasi yang dimilikinya adalah milik Jurusan Informatika-Telkom dan bersifat rahasia. Dilarang me-reproduksi dokumen ini tanpa diketahui oleh Jurusan Informatika-Telkom.		

2. **Skema Validasi Masukan dan Keluaran (*Pydantic Schemas*)**: Menggunakan kelas berbasis Pydantic di FastAPI untuk menyaring, memvalidasi tipe data secara ketat, serta memastikan struktur payload JSON yang dikirimkan dari antarmuka Vue.js sudah sesuai sebelum diproses oleh fungsi logika bisnis peladen.

4.3 Perancangan Interaksi Proses (*Sequence Diagram*)



Gambar 4.3.1 Sequence Diagram

Menjelaskan kronologi urutan pesan interaksi antar komponen secara kronologis berdasarkan waktu untuk skenario fitur-fitur utama sistem:

1. Skenario Unggah dan Validasi Pembayaran Sewa:

- Penghuni memilih menu tagihan pada antarmuka Vue.js dan mengunggah berkas bukti transfer.
- Aplikasi Vue.js mengirimkan permintaan HTTP POST beserta token JWT dan berkas lampiran ke *endpoint* transaksi FastAPI.
- FastAPI memvalidasi token, menyimpan berkas lampiran melalui integrasi Google Drive API, lalu memperbarui status baris data pada tabel *transactions* di PostgreSQL menjadi *Menunggu Verifikasi*.
- Pemilik kos membuka dasbor, melihat daftar transaksi tertunda, lalu melakukan konfirmasi validasi pembayaran.
- FastAPI memproses perubahan status menjadi *Lunas*, memperbarui masa sewa pada tabel *leases*, dan mengirimkan respons JSON kembali ke Vue.js untuk merender status tagihan hijau secara *real-time*.

2. Skenario Pengajuan dan Persetujuan Surat Izin Keluar Masuk:

- Penghuni mengisi formulir izin pada aplikasi Vue.js yang kemudian mengirimkan permintaan POST ke *endpoint* perizinan FastAPI.
- FastAPI memvalidasi data masukan dan menyimpannya ke tabel *permissions* di PostgreSQL dengan status awal *Pending*.
- Sistem mengirimkan pemberitahuan kepada Pemilik Kos. Pemilik kos meninjau permohonan melalui dasbor lalu memutuskan untuk menyetujui atau menolak perizinan.
- Peladen FastAPI memperbarui status izin di basis data dan mengembalikan respons status keputusan kepada aplikasi penghuni.

BAB V

KESIMPULAN

Berdasarkan keseluruhan rangkaian perancangan perangkat lunak yang telah dilakukan untuk aplikasi Kostopia, dapat disimpulkan bahwa:

1. **Arsitektur Sistem yang Terstruktur:** Aplikasi Kostopia dirancang menggunakan pendekatan arsitektur tiga tingkat (*3-tier architecture*) yang memisahkan secara jelas antara antarmuka pengguna berbasis *Vue.js*, logika peladen berbasis kerangka kerja *FastAPI*, serta penyimpanan data relasional menggunakan *PostgreSQL*.
2. **Pemodelan Data dan Proses yang Komprehensif:** Perancangan sistem telah dilengkapi dengan pemodelan yang matang, mulai dari pemetaan hubungan entitas data (*ERD*), penjabaran alur data berjenjang (*DFD Level 0 hingga Level 2*), hingga rincian pemodelan tabel menggunakan *SQLAlchemy ORM* dan validasi skema *Pydantic*.
3. **Kesiapan Implementasi Teknis:** Spesifikasi teknis antarmuka pemrograman aplikasi (*RESTful API Endpoints*), mekanisme pengelolaan berkas eksternal melalui *Google Drive API*, manajemen repositori via *GitHub*, serta pengujian skenario alur kerja melalui *Sequence Diagram* memastikan bahwa cetak biru perangkat lunak ini siap untuk dilanjutkan ke tahap implementasi pengkodean dan pengujian sistem secara nyata.